

Getting Started with Q2NS using Docker

1 Overview

This guide provides a simple walkthrough for using a Docker-based environment for installing and running Q2NS. The goal is to provide a reproducible setup that avoids most operating-system-specific configuration and dependency issues.

The Docker configuration in this guide creates a reproducible Linux environment containing ns-3 and Q2NS. After you build the image once, you can run the provided examples, create custom simulations, and generate traces for visualization with Q2NSViz without configuring the ns-3 build environment directly on your host system.

This guide is intended primarily for learning, experimentation, and running the provided tutorials.

2 Prerequisites

Before beginning, install:

- Git
- Docker

For Windows and macOS, Docker Desktop is the recommended installation method.

For Linux, you can use either Docker Desktop or Docker Engine.

Official Docker installation instructions are available at:

https://docs.docker.com/get-docker/

After installation, verify that Docker is available:

```
docker --version
```

Optionally, verify that Docker can successfully start a container:

```
docker run hello-world
```

3 Download Q2NS

Clone the Q2NS repository:

```
git clone https://github.com/QuantumInternet-it/q2ns.git
cd q2ns
```

The following steps create a small Docker configuration locally in the Q2NS repository. You do not need to modify the Q2NS source code.

4 Create the Docker Configuration

From the root directory of the Q2NS repository, create a file named `Dockerfile` with the following contents:

```
FROM ubuntu:24.04

ENV DEBIAN_FRONTEND=noninteractive

RUN apt-get update && apt-get install -y \
    build-essential \
    cmake \
    git \
    ninja-build \
    python3 \
    python3-pip \
    wget \
    && rm -rf /var/lib/apt/lists/*

WORKDIR /workspace

RUN git clone --branch ns-3.47 --depth 1 \
    https://gitlab.com/nsnam/ns-3-dev.git ns-3

COPY . /workspace/ns-3/contrib/q2ns

WORKDIR /workspace/ns-3

RUN ./ns3 configure --enable-examples \
    && ./ns3 build -j 2

CMD ["/bin/bash"]
```

Next, create a file named `.dockerignore` in the same directory:

```
.git
__pycache__
*.pyc
```

The Dockerfile creates an Ubuntu-based environment, downloads ns-3.47, copies your local Q2NS source tree into `contrib/q2ns`, and builds ns-3 and Q2NS. The build is limited to two parallel

compilation jobs to reduce memory usage on systems with constrained Docker resources.

5 Build the Docker Image

From the root directory of the Q2NS repository, where you created the `Dockerfile`, run:

```
docker build -t q2ns .
```

This creates a local Docker image named `q2ns`. The first build may take several minutes because Docker must download the base environment and compile ns-3 and Q2NS. Subsequent builds can reuse previously downloaded and compiled components when possible, so they will not take as long.

6 Start Q2NS

Create and enter a Q2NS container with:

```
docker run -it --name q2ns q2ns
```

After entering the container, your working directory should be:

```
/workspace/ns-3
```

You can verify this with:

```
pwd
```

7 Run a First Q2NS Example

From inside the container, run:

```
./ns3 run q2ns-1-basics-example
```

This example provides a basic check that ns-3 and Q2NS were successfully built and that your simulation environment is working correctly.

You can find additional Q2NS examples in:

```
contrib/q2ns/examples/
```

8 Exit and Restart the Container

To leave the Q2NS container, run:

```
exit
```

This stops the container *but does not delete it.* You do not need to rebuild the Docker image each time.

To return to the same Q2NS environment later, run:

```
docker start -ai q2ns
```

For reference, the flags mean:

- -a = –attach — attach your terminal to the container's standard output/error
- -i = –interactive — attach standard input too, so you can type into the container

9 Reset the Container

If you need to recreate the container, first remove the existing container:

```
docker rm q2ns
```

Then create a new one from the existing image:

```
docker run -it --name q2ns q2ns
```

The Docker image itself remains available unless you explicitly remove it.

10 Write a Custom Q2NS Simulation

You can also write and run custom Q2NS simulations directly inside the Docker container.

From the ns-3 root directory, create a new C++ source file in the `scratch/` directory. For example:

```
scratch/q2ns-custom-sim.cc
```

A minimal Q2NS simulation can follow the same structure as the provided examples. For example:

```
#include "ns3/core-module.h"
#include "ns3/q2ns-netcontroller.h"
#include "ns3/q2ns-qgate.h"
#include "ns3/q2ns-qnode.h"
#include "ns3/q2ns-qstate.h"

using namespace ns3;
using namespace q2ns;

int main() {
    RngSeedManager::SetSeed(42);
    RngSeedManager::SetRun(1);

    NetController net;
    net.SetQStateBackend(QStateBackend::Ket);

    auto node = net.CreateNode();
    auto q = node->CreateQubit();

    Simulator::Schedule(MicroSeconds(10), [node, q]() {
        node->Apply(gates::H(), {q});
        std::cout << "State after H: " << node->GetState(q) << "\n";
    });

    Simulator::Schedule(MicroSeconds(20), [node, q]() {
        int result = node->Measure(q);
        std::cout << "Measurement result: " << result << "\n";
    });

    Simulator::Stop(MilliSeconds(10));
    Simulator::Run();
    Simulator::Destroy();
    return 0;
}
```

Run the simulation with:

```
./ns3 run scratch/q2ns-custom-sim
```

Files placed in `scratch/` are automatically detected by ns-3, making this a convenient way for you to experiment with Q2NS without modifying the provided examples or creating new build configurations.

11 Visualize a Q2NS Simulation

Q2NS can generate trace files that can be opened with the standalone Q2NSViz desktop application. Because Q2NS runs inside Docker while Q2NSViz runs on your host computer, you can use a shared directory to make generated traces available to both.

First, exit the Q2NS container if it is currently running:

```
exit
```

From the Q2NS repository on your host computer, remove the existing container:

```
docker rm q2ns
```

Create a directory for visualization traces:

```
mkdir traces
```

Then create the container again with this directory mounted to the Q2NS example trace directory:

```
docker run -it --name q2ns -v "${PWD}/traces:/workspace/ns-3/contrib/q2ns/examples/
    example_traces" q2ns
```

The host `traces/` directory is now mounted at `/workspace/ns-3/contrib/q2ns/examples/example_traces` inside the container. Files written there by Q2NS therefore appear immediately in `traces/` on your host computer.

From inside the container, run a visualization-enabled example:

```
./ns3 run q2nsviz-teleportation-example
```

The generated trace will now also appear in the `traces/` directory on your host computer.

Install Q2NSViz

Q2NSViz runs directly on your host computer rather than inside Docker. To install it, open a separate host terminal and run:

```
git clone https://github.com/QuantumInternet-it/q2nsviz.git
cd q2nsviz
python -m pip install .
```

After installation, you can launch Q2NSViz with:

```
q2nsviz
```

You can then select a trace from the shared `traces/` directory using the Q2NSViz interface.

Alternatively, you can open a trace directly from the command line:

```
q2nsviz path/to/traces/q2nsviz-teleportation-example.json
```

You only need to configure this mount when the container is created. Afterward, you can stop and restart the same container normally with `docker start -ai q2ns`.

12 Next Steps

After confirming that the basic examples work, you can explore the additional simulations provided in:

```
contrib/q2ns/examples/
```

Q2NS source code, examples, and project information are available at:

https://github.com/QuantumInternet-it/q2ns

The Q2NS entry in the ns-3 App Store is available at:

https://apps.nsnam.org/app/q2ns/

General ns-3 documentation is available at:

https://www.nsnam.org/documentation/

A future development-oriented workflow may additionally support editing Q2NS simulations directly from your host system or through development environments such as Visual Studio Code. The Docker setup described here focuses on providing you with a simple and reproducible environment for getting started with Q2NS.

Q2NS Resources

Q2NS GitHub Repository

Q2NS on the ns-3 App Store